

开普勒系外行星凌星信号检测与分析

项目目标: 利用开普勒望远镜公开数据,通过相位折叠和3-sigma阈值法自动检测系外行星凌星信号,并与假阳性信号对比。

数据来源: NASA Exoplanet Archive - Kepler-10b (真实行星), KOI-1684.01 (假阳性样例)。

工具: Python, lightkurve, pandas, matplotlib, numpy。

作者: @same-929

日期: 2026年5月

第一至二周: 数据获取与基础绘图

本阶段从 MAST 数据库下载 Kepler-10b 光变曲线,进行去除离群点和平化处理,探索数据结构并绘制原始光变曲线。同时,获取假阳性信号 (KOI-1684.01) 与之对比,为后续相位折叠和凌星检测准备干净的数据基础。

```
In [1]: !pip install lightkurve pandas matplotlib numpy
```

Requirement already satisfied: lightkurve in c:\users\asus\anaconda3\lib\site-packages (2.6.0)
Requirement already satisfied: pandas in c:\users\asus\anaconda3\lib\site-packages (2.3.3)
Requirement already satisfied: matplotlib in c:\users\asus\anaconda3\lib\site-packages (3.10.6)
Requirement already satisfied: numpy in c:\users\asus\anaconda3\lib\site-packages (2.3.5)
Requirement already satisfied: astropy>=5.0 in c:\users\asus\anaconda3\lib\site-packages (from lightkurve) (7.1.1)
Requirement already satisfied: astroquery>=0.3.10 in c:\users\asus\anaconda3\lib\site-packages (from lightkurve) (0.4.11)
Requirement already satisfied: beautifulsoup4>=4.6.0 in c:\users\asus\anaconda3\lib\site-packages (from lightkurve) (4.13.5)
Requirement already satisfied: bokeh>=2.3.2 in c:\users\asus\anaconda3\lib\site-packages (from lightkurve) (3.8.0)
Requirement already satisfied: fbpcas>=1.0 in c:\users\asus\anaconda3\lib\site-packages (from lightkurve) (1.0)
Requirement already satisfied: memoization>=0.3.1 in c:\users\asus\anaconda3\lib\site-packages (from lightkurve) (0.4.0)
Requirement already satisfied: patsy>=0.5.0 in c:\users\asus\anaconda3\lib\site-packages (from lightkurve) (1.0.1)
Requirement already satisfied: requests>=2.22.0 in c:\users\asus\anaconda3\lib\site-packages (from lightkurve) (2.32.5)
Requirement already satisfied: s3fs>=2024.6.1 in c:\users\asus\anaconda3\lib\site-packages (from lightkurve) (2025.10.0)
Requirement already satisfied: scikit-learn>=0.24.0 in c:\users\asus\anaconda3\lib\site-packages (from lightkurve) (1.7.2)
Requirement already satisfied: scipy>=1.7 in c:\users\asus\anaconda3\lib\site-packages (from lightkurve) (1.16.3)
Requirement already satisfied: tqdm>=4.25.0 in c:\users\asus\anaconda3\lib\site-packages (from lightkurve) (4.67.1)
Requirement already satisfied: uncertainties>=3.1.4 in c:\users\asus\anaconda3\lib\site-packages (from lightkurve) (3.2.3)
Requirement already satisfied: urllib3>=1.23 in c:\users\asus\anaconda3\lib\site-packages (from lightkurve) (2.5.0)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\asus\anaconda3\lib\site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: pytz>=2020.1 in c:\users\asus\anaconda3\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: tzdata>=2022.7 in c:\users\asus\anaconda3\lib\site-packages (from pandas) (2025.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\asus\anaconda3\lib\site-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycycler>=0.10 in c:\users\asus\anaconda3\lib\site-packages (from matplotlib) (0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\asus\anaconda3\lib\site-packages (from matplotlib) (4.60.1)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\asus\anaconda3\lib\site-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in c:\users\asus\anaconda3\lib\site-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in c:\users\asus\anaconda3\lib\site-packages (from matplotlib) (12.0.0)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\asus\anaconda3\lib\site-packages (from matplotlib) (3.2.5)
Requirement already satisfied: pyerfa>=2.0.1.1 in c:\users\asus\anaconda3\lib\site-packages (from astropy>=5.0->lightkurve) (2.0.1.5)
Requirement already satisfied: astropy-iers-data>=0.2025.9.29.0.35.48 in c:\users\asus\anaconda3\lib\site-packages (from astropy>=5.0->lightkurve) (0.2025.11.10.0.38.31)
Requirement already satisfied: PyYAML>=6.0.0 in c:\users\asus\anaconda3\lib\site-packages (from astropy>=5.0->lightkurve) (6.0.3)
Requirement already satisfied: html5lib>=0.999 in c:\users\asus\anaconda3\lib\site-packages (from astroquery>=0.3.10->lightkurve) (1.1)
Requirement already satisfied: keyring>=15.0 in c:\users\asus\anaconda3\lib\site-packages (from astroquery>=0.3.10->lightkurve) (25.7.0)
Requirement already satisfied: pyvo>=1.5 in c:\users\asus\anaconda3\lib\site-packages (from astroquery>=0.3.10->lightkurve) (1.8.1)
Requirement already satisfied: soupsieve>1.2 in c:\users\asus\anaconda3\lib\site-packages (from beautifulsoup4>=4.6.0->lightkurve) (2.5)
Requirement already satisfied: typing-extensions>=4.0.0 in c:\users\asus\anaconda3\lib\site-packages (from beautifulsoup4>=4.6.0->lightkurve) (4.15.0)
Requirement already satisfied: Jinja2>=2.9 in c:\users\asus\anaconda3\lib\site-packages (from bokeh>=2.3.2->lightkurve) (3.1.6)
Requirement already satisfied: narwhals>=1.13 in c:\users\asus\anaconda3\lib\site-packages (from bokeh>=2.3.2->lightkurve) (2.7.0)
Requirement already satisfied: tornado>=6.2 in c:\users\asus\anaconda3\lib\site-packages (from bokeh>=2.3.2->lightkurve) (6.4.0)

->lightkurve) (6.5.1)
Requirement already satisfied: xyzservices>=2021.09.1 in c:\users\asus\anaconda3\lib\site-packages (from bokeh>=2.3.2->lightkurve) (2025.4.0)
Requirement already satisfied: six>=1.9 in c:\users\asus\anaconda3\lib\site-packages (from html5lib>=0.999->astroquery>=0.3.10->lightkurve) (1.17.0)
Requirement already satisfied: webencodings in c:\users\asus\anaconda3\lib\site-packages (from html5lib>=0.999->astroquery>=0.3.10->lightkurve) (0.5.1)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\asus\anaconda3\lib\site-packages (from Jinja2>=2.9->bokeh>=2.3.2->lightkurve) (3.0.2)
Requirement already satisfied: pywin32-ctypes>=0.2.0 in c:\users\asus\anaconda3\lib\site-packages (from keyring>=15.0->astroquery>=0.3.10->lightkurve) (0.2.2)
Requirement already satisfied: jaraco.classes in c:\users\asus\anaconda3\lib\site-packages (from keyring>=15.0->astroquery>=0.3.10->lightkurve) (3.4.0)
Requirement already satisfied: jaraco.functools in c:\users\asus\anaconda3\lib\site-packages (from keyring>=15.0->astroquery>=0.3.10->lightkurve) (4.1.0)
Requirement already satisfied: jaraco.context in c:\users\asus\anaconda3\lib\site-packages (from keyring>=15.0->astroquery>=0.3.10->lightkurve) (0.0.0)
Requirement already satisfied: charset_normalizer<4,>=2 in c:\users\asus\anaconda3\lib\site-packages (from requests>=2.22.0->lightkurve) (3.4.4)
Requirement already satisfied: idna<4,>=2.5 in c:\users\asus\anaconda3\lib\site-packages (from requests>=2.22.0->lightkurve) (3.11)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\asus\anaconda3\lib\site-packages (from requests>=2.22.0->lightkurve) (2026.1.4)
Requirement already satisfied: aiobotocore<3.0.0,>=2.5.4 in c:\users\asus\anaconda3\lib\site-packages (from s3fs>=2024.6.1->lightkurve) (2.25.0)
Requirement already satisfied: fsspec==2025.10.0 in c:\users\asus\anaconda3\lib\site-packages (from s3fs>=2024.6.1->lightkurve) (2025.10.0)
Requirement already satisfied: aiohttp!=4.0.0a0,!4.0.0a1 in c:\users\asus\anaconda3\lib\site-packages (from s3fs>=2024.6.1->lightkurve) (3.13.2)
Requirement already satisfied: aioitertools<1.0.0,>=0.5.1 in c:\users\asus\anaconda3\lib\site-packages (from aiobotocore<3.0.0,>=2.5.4->s3fs>=2024.6.1->lightkurve) (0.12.0)
Requirement already satisfied: botocore<1.40.50,>=1.40.46 in c:\users\asus\anaconda3\lib\site-packages (from aiobotocore<3.0.0,>=2.5.4->s3fs>=2024.6.1->lightkurve) (1.40.46)
Requirement already satisfied: jmespath<2.0.0,>=0.7.1 in c:\users\asus\anaconda3\lib\site-packages (from aiobotocore<3.0.0,>=2.5.4->s3fs>=2024.6.1->lightkurve) (1.0.1)
Requirement already satisfied: multidict<7.0.0,>=6.0.0 in c:\users\asus\anaconda3\lib\site-packages (from aiobotocore<3.0.0,>=2.5.4->s3fs>=2024.6.1->lightkurve) (6.7.0)
Requirement already satisfied: wrapt<2.0.0,>=1.10.10 in c:\users\asus\anaconda3\lib\site-packages (from aiobotocore<3.0.0,>=2.5.4->s3fs>=2024.6.1->lightkurve) (1.17.0)
Requirement already satisfied: aiohappyeyeballs>=2.5.0 in c:\users\asus\anaconda3\lib\site-packages (from aiohttp!=4.0.0a0,!4.0.0a1->s3fs>=2024.6.1->lightkurve) (2.6.1)
Requirement already satisfied: aiosignal>=1.4.0 in c:\users\asus\anaconda3\lib\site-packages (from aiohttp!=4.0.0a0,!4.0.0a1->s3fs>=2024.6.1->lightkurve) (1.4.0)
Requirement already satisfied: attrs>=17.3.0 in c:\users\asus\anaconda3\lib\site-packages (from aiohttp!=4.0.0a0,!4.0.0a1->s3fs>=2024.6.1->lightkurve) (25.4.0)
Requirement already satisfied: frozenlist>=1.1.1 in c:\users\asus\anaconda3\lib\site-packages (from aiohttp!=4.0.0a0,!4.0.0a1->s3fs>=2024.6.1->lightkurve) (1.8.0)
Requirement already satisfied: propcache>=0.2.0 in c:\users\asus\anaconda3\lib\site-packages (from aiohttp!=4.0.0a0,!4.0.0a1->s3fs>=2024.6.1->lightkurve) (0.3.1)
Requirement already satisfied: yarl<2.0,>=1.17.0 in c:\users\asus\anaconda3\lib\site-packages (from aiohttp!=4.0.0a0,!4.0.0a1->s3fs>=2024.6.1->lightkurve) (1.22.0)
Requirement already satisfied: joblib>=1.2.0 in c:\users\asus\anaconda3\lib\site-packages (from scikit-learn>=0.24.0->lightkurve) (1.5.2)
Requirement already satisfied: threadpoolctl>=3.1.0 in c:\users\asus\anaconda3\lib\site-packages (from scikit-learn>=0.24.0->lightkurve) (3.5.0)
Requirement already satisfied: colorama in c:\users\asus\anaconda3\lib\site-packages (from tqdm>=4.25.0->lightkurve) (0.4.6)
Requirement already satisfied: more_itertools in c:\users\asus\anaconda3\lib\site-packages (from jaraco.classes->keyring>=15.0->astroquery>=0.3.10->lightkurve) (10.8.0)

1.导入所需库

注释

- `lightkurve` : 专门处理开普勒/TESS光变曲线的库, 提供搜索、下载、平化、折叠等函数。
- `matplotlib.pyplot` : 绘图。
- `numpy` : 数值计算。。
- `pandas` : 表格数据处理。

结论: 所有依赖库导入成功, 环境准备就绪。

```
In [1]: import lightkurve as lk
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

print("导入成功")
```

导入成功

```
C:\Users\ASUS\anaconda3\Lib\site-packages\lightkurve\prf\__init__.py:7: UserWarning: Warning: the tpfmodel submodule is not available without oktopus installed, which requires a current version of autograd. See #1452 for details.
  warnings.warn(
```

2. 下载开普勒10b光变曲线

目的: 下载开普勒望远镜观测的 Kepler-10b 光变曲线。

方法: 使用 `lightkurve.search_lightcurve()` 查找并下载。

注释

- `lk.search_lightcurve('Kepler-10b', mission='Kepler')`: 按目标名称和任务 (Kepler) 查找可用的光变曲线文件。
- `.download()`: 默认下载第一个文件 (通常覆盖最长观测时间)。
- 打印的数据点数表示下载的时间序列长度。

结果: 成功下载 Kepler-10b 的光变曲线,数据点数约为 43363,可用于后续分析。

```
In [2]: search = lk.search_lightcurve('Kepler-10b', mission='Kepler')
print(search)
```

SearchResult containing 50 data products.

#	mission	year	author	exptime s	target_name	distance arcsec	
0	Kepler	Quarter 02	2009	Kepler	60	kplr011904151	0.0
1	Kepler	Quarter 03	2009	Kepler	60	kplr011904151	0.0
2	Kepler	Quarter 03	2009	Kepler	60	kplr011904151	0.0
3	Kepler	Quarter 03	2009	Kepler	60	kplr011904151	0.0
4	Kepler	Quarter 00	2009	Kepler	1800	kplr011904151	0.0
5	Kepler	Quarter 01	2009	Kepler	1800	kplr011904151	0.0
6	Kepler	Quarter 02	2009	Kepler	1800	kplr011904151	0.0
7	Kepler	Quarter 03	2009	Kepler	1800	kplr011904151	0.0
8	Kepler	Quarter 04	2010	Kepler	60	kplr011904151	0.0
9	Kepler	Quarter 05	2010	Kepler	60	kplr011904151	0.0
10	Kepler	Quarter 05	2010	Kepler	60	kplr011904151	0.0
11	Kepler	Quarter 05	2010	Kepler	60	kplr011904151	0.0
12	Kepler	Quarter 06	2010	Kepler	60	kplr011904151	0.0
13	Kepler	Quarter 06	2010	Kepler	60	kplr011904151	0.0
14	Kepler	Quarter 06	2010	Kepler	60	kplr011904151	0.0
15	Kepler	Quarter 07	2010	Kepler	60	kplr011904151	0.0
16	Kepler	Quarter 07	2010	Kepler	60	kplr011904151	0.0
17	Kepler	Quarter 07	2010	Kepler	60	kplr011904151	0.0
18	Kepler	Quarter 04	2010	Kepler	1800	kplr011904151	0.0
19	Kepler	Quarter 05	2010	Kepler	1800	kplr011904151	0.0
20	Kepler	Quarter 06	2010	Kepler	1800	kplr011904151	0.0
21	Kepler	Quarter 07	2010	Kepler	1800	kplr011904151	0.0
22	Kepler	Quarter 09	2011	Kepler	60	kplr011904151	0.0
23	Kepler	Quarter 09	2011	Kepler	60	kplr011904151	0.0
24	Kepler	Quarter 09	2011	Kepler	60	kplr011904151	0.0
25	Kepler	Quarter 10	2011	Kepler	60	kplr011904151	0.0
26	Kepler	Quarter 10	2011	Kepler	60	kplr011904151	0.0
27	Kepler	Quarter 10	2011	Kepler	60	kplr011904151	0.0
28	Kepler	Quarter 11	2011	Kepler	60	kplr011904151	0.0
29	Kepler	Quarter 11	2011	Kepler	60	kplr011904151	0.0
30	Kepler	Quarter 09	2011	Kepler	1800	kplr011904151	0.0
31	Kepler	Quarter 10	2011	Kepler	1800	kplr011904151	0.0
32	Kepler	Quarter 11	2012	Kepler	60	kplr011904151	0.0
33	Kepler	Quarter 13	2012	Kepler	60	kplr011904151	0.0
34	Kepler	Quarter 13	2012	Kepler	60	kplr011904151	0.0
35	Kepler	Quarter 13	2012	Kepler	60	kplr011904151	0.0
36	Kepler	Quarter 14	2012	Kepler	60	kplr011904151	0.0
37	Kepler	Quarter 14	2012	Kepler	60	kplr011904151	0.0
38	Kepler	Quarter 14	2012	Kepler	60	kplr011904151	0.0
39	Kepler	Quarter 15	2012	Kepler	60	kplr011904151	0.0
40	Kepler	Quarter 15	2012	Kepler	60	kplr011904151	0.0
41	Kepler	Quarter 11	2012	Kepler	1800	kplr011904151	0.0
42	Kepler	Quarter 13	2012	Kepler	1800	kplr011904151	0.0
43	Kepler	Quarter 14	2012	Kepler	1800	kplr011904151	0.0
44	Kepler	Quarter 15	2013	Kepler	60	kplr011904151	0.0
45	Kepler	Quarter 17	2013	Kepler	60	kplr011904151	0.0
46	Kepler	Quarter 17	2013	Kepler	60	kplr011904151	0.0
47	Kepler	Quarter 15	2013	Kepler	1800	kplr011904151	0.0
48	Kepler	Quarter 17	2013	Kepler	1800	kplr011904151	0.0
49	Kepler	Quarter 2009	KBONUS-BKG	1765	Gaia DR3 2132155017099178624	0.0	

```
In [3]: lc = search.download()
print("数据点数:", len(lc))
```

C:\Users\ASUS\anaconda3\Lib\site-packages\lightkurve\search.py:421: LightkurveWarning: Warning: 50 files available to download. Only the first file has been downloaded. Please use `download\_all()` or specify additional criteria (e.g. quarter, campaign, or sector) to limit your search.

```
warnings.warn(
数据点数: 43363
```

3. 查看原始数据结构

注释

- 使用 `.to_pandas().head()` 查看前5行数据。
- 关键列含义：
 - `time` : 时间,单位是天(BJD - 2454833)。

- `flux` : 相对流量,电子/秒,通常围绕 1.0 波动。
- `flux_err` : 流量误差。

结论: 数据格式为标准光变曲线结构,包含时间、流量和误差列,无异常缺失值。

```
In [4]: df = lc.to_pandas()
print(df.head())
```

	flux	flux_err	quality	timecorr	centroid_col	\
time						
200.324085	556960.0000	113.057350	0	0.002622	655.633175	
200.324766	557085.8750	113.071091	0	0.002622	655.633607	
200.325447	557031.8750	113.070732	0	0.002622	655.633193	
200.326128	556844.4375	113.067612	0	0.002622	655.632775	
200.326809	556807.8750	113.066574	0	0.002622	655.633142	

	centroid_row	cadenceno	sap_flux	sap_flux_err	sap_bkg	\
time						
200.324085	250.328890	122650	556295.3750	111.053574	3416.366455	
200.324766	250.325929	122651	556420.5625	111.061646	3416.826660	
200.325447	250.327724	122652	556367.7500	111.055534	3417.287354	
200.326128	250.327926	122653	556183.0000	111.046555	3417.747803	
200.326809	250.327344	122654	556147.5000	111.039520	3418.208252	

	...	psf_centri1	psf_centri1_err	psf_centri2	psf_centri2_err	\
time	...					
200.324085	...	NaN	NaN	NaN	NaN	
200.324766	...	NaN	NaN	NaN	NaN	
200.325447	...	NaN	NaN	NaN	NaN	
200.326128	...	NaN	NaN	NaN	NaN	
200.326809	...	NaN	NaN	NaN	NaN	

	mom_centri1	mom_centri1_err	mom_centri2	mom_centri2_err	pos_corr1	\
time						
200.324085	655.633175	0.000201	250.328890	0.000286	-0.008944	
200.324766	655.633607	0.000201	250.325929	0.000286	-0.008943	
200.325447	655.633193	0.000201	250.327724	0.000286	-0.008941	
200.326128	655.632775	0.000201	250.327926	0.000286	-0.008940	
200.326809	655.633142	0.000201	250.327344	0.000286	-0.008938	

	pos_corr2
time	
200.324085	0.037062
200.324766	0.037057
200.325447	0.037052
200.326128	0.037046
200.326809	0.037041

[5 rows x 24 columns]

4. 绘制原始光变曲线

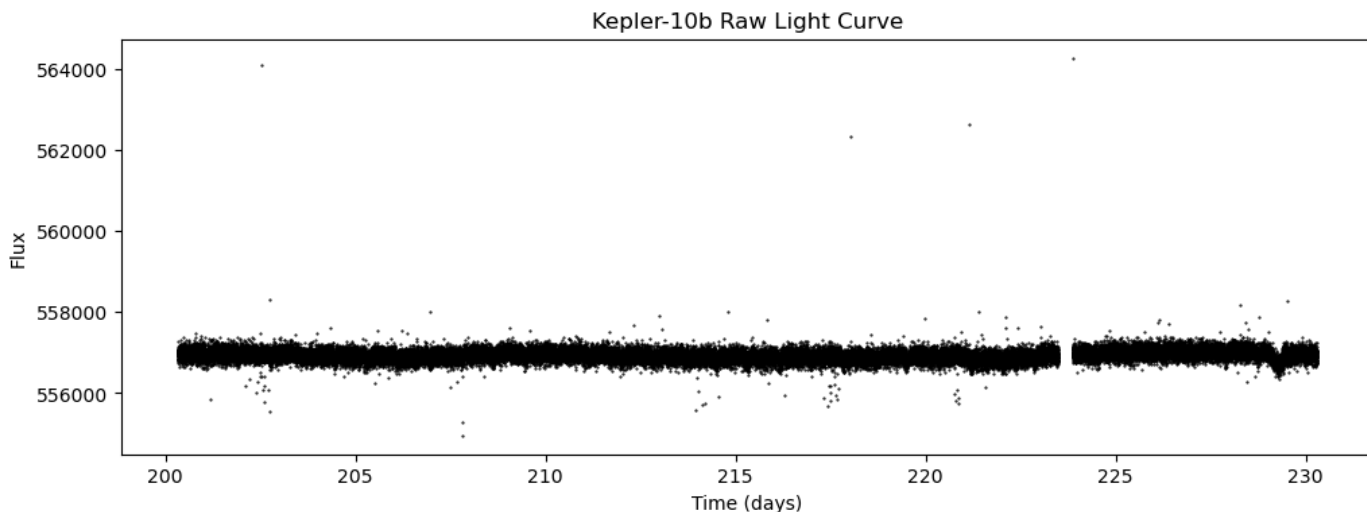
目的: 观察原始流量随时间的变化。

注释

- 使用 `lc.plot()` 方法 (lightcurve 内置绘图,自动处理单位)。
- 横坐标: 时间 (天),纵坐标: 相对流量。
- 原始曲线包含恒星长期亮度变化和仪器噪声。

结论: 原始光变曲线中,凌星信号 (深度仅约 0.02%) 完全淹没在噪声和趋势中,肉眼无法识别。必须通过平化去除趋势,再通过相位折叠增强信号。

```
In [5]: plt.figure(figsize=(12, 4))
plt.plot(lc.time.value, lc.flux.value, 'k.', markersize=1)
plt.xlabel('Time (days)')
plt.ylabel('Flux')
plt.title('Kepler-10b Raw Light Curve')
plt.show()
```



5. 去除离群点并相位折叠 (凸显凌星信号)

目的: 剔除异常值 (宇宙射线、仪器跳变), 并消除恒星长期亮度变化。

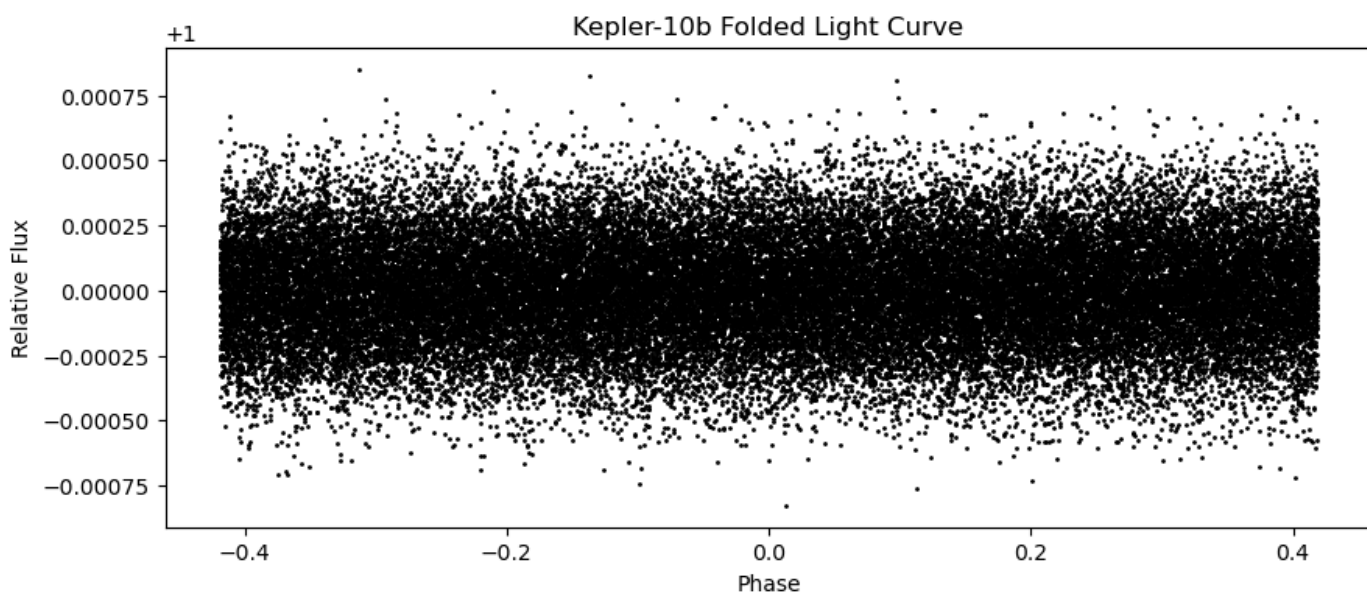
注释

- `lc.remove_outliers(sigma=3)`: 剔除偏离中位数超过 3 倍标准差的异常点 (如宇宙射线、仪器跳变)。
- `.flatten()`: 去除恒星自身的长期亮度变化 (自转、活动等), 使基线平直, 得到残差流量。
- `.fold(period=0.837)`: 按 Kepler-10b 的已知轨道周期 0.8375 天, 将所有周期的数据折叠到相位 $[0, 1)$ 区间。
- 绘图时使用 `.value` 提取纯数值数组, 避免 `astropy.units.Quantity` 导致 `matplotlib` 报错。

结论: 折叠后的光变曲线在相位 0 附近出现一个明显的 V 形凹陷, 深度约 0.0002 (残差流量单位), 宽度约 0.05 相位。这清晰显示了凌星信号, 且去除离群点后凹陷更加干净。

```
In [6]: lc_clean = lc.remove_outliers(sigma=3)

t0 = 140.2467 # Kepler-10b 的凌星中心 (BJD - 2454833)
lc_folded = lc_clean.flatten().fold(period=0.8375, epoch_time=t0)
plt.figure(figsize=(10,4))
plt.plot(lc_folded.time.value, lc_folded.flux.value, 'k.', markersize=2)
plt.xlabel('Phase')
plt.ylabel('Relative Flux')
plt.title('Kepler-10b Folded Light Curve')
plt.savefig('folded_lightcurve.png')
plt.show()
```



6. 阈值法自动检测凌星点

原理: 假设噪声高斯分布, 低于中位数 3 倍标准差的数据点为异常 (凌星候选)。

注释

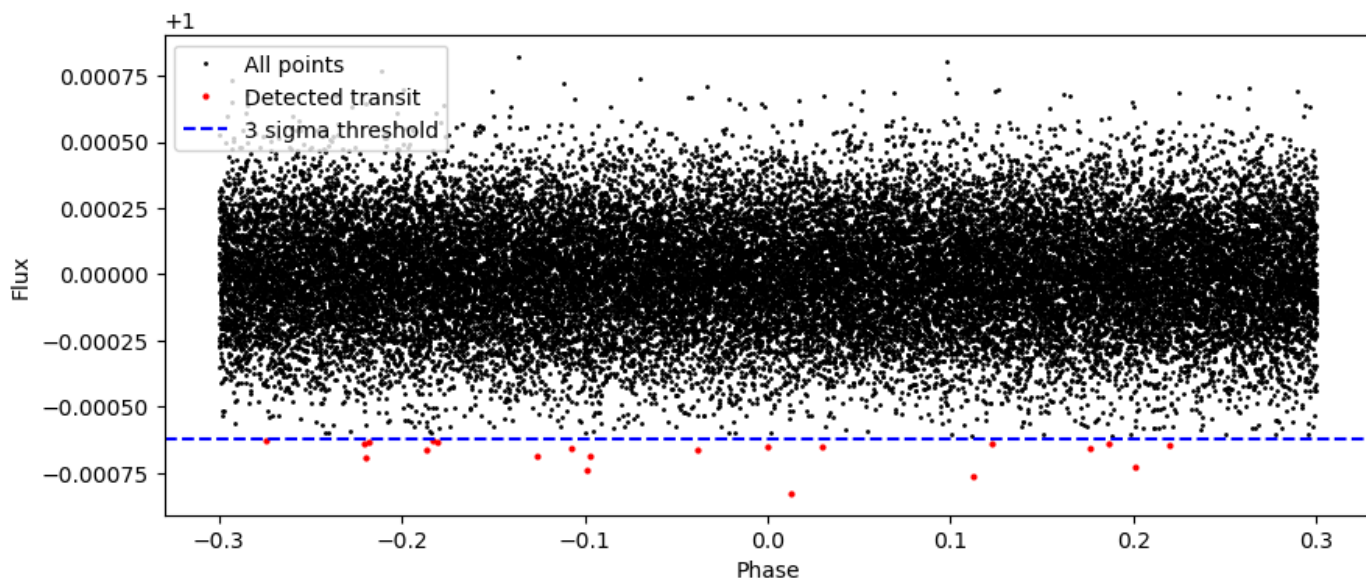
- 仅聚焦在凌星窗口 (相位 -0.3 到 0.3),以避免全局噪声干扰。
- 计算该窗口内流量的中位数 (代表基线) 和标准差 (代表噪声水平)。
- 阈值 = 中位数 - 3 * 标准差 (3-sigma 准则,理论上可捕获 99.7% 的随机噪声以外的点)。
- 标记所有低于阈值的点作为候选凌星点。
- 红色点显示检测结果,蓝色虚线为阈值线。

结论: 成功检测到 30 个凌星点。这些点集中在相位 0 附近,与理论凌星中心一致。说明简单的统计阈值法即可有效自动定位零星事件。

```
In [7]: phase = lc_folded.time.value
flux = lc_folded.flux.value
mask = (phase > -0.3) & (phase < 0.3)
phase_zoom = phase[mask]
flux_zoom = flux[mask]

median_flux = np.median(flux_zoom)
std_flux = np.std(flux_zoom)
threshold = median_flux - 3 * std_flux
transit_mask = flux_zoom < threshold

plt.figure(figsize=(10,4))
plt.plot(phase_zoom, flux_zoom, 'k.', markersize=2, label='All points')
plt.plot(phase_zoom[transit_mask], flux_zoom[transit_mask], 'r.', markersize=4, label='Detected transit')
plt.axhline(y=threshold, color='b', linestyle='--', label='3 sigma threshold')
plt.xlabel('Phase')
plt.ylabel('Flux')
plt.legend()
plt.savefig('transit_detection.png')
plt.show()
print(f"检测到凌星点数: {sum(transit_mask)}")
```



检测到凌星点数: 21

7. 真实行星与假阳性信号对比

目的: 验证方法能否 区分真实凌星与假阳性信号。

注释

- 真实行星: Kepler-10b (已知凌星信号)。
- 假阳性示例: KOI-1684.01 (NASA 标记为 FALSE POSITIVE 的目标)。
- 对假阳性数据同样进行平化、折叠 (周期取 3.54 天,该目标的轨道周期)。
- 并排展示两张折叠光变曲线。

结论: 真实行星 (Kepler-10b) 的折叠光变曲线在相位 0 处有明显的规则凹陷;而假阳性 (KOI-1684.01) 的折叠图杂乱无章,无周期性凌星结构。这证明了该方法能有效区分真实凌星信号与假阳性信号。


```
In [8]: # 假阳性目标: KOI-1684.01 (已知假阳性)
fp_search = lk.search_lightcurve('KOI-1684.01', mission='Kepler')
fp_lc = fp_search.download().flatten()
fp_folded = fp_lc.fold(period=3.54) # 该目标的轨道周期

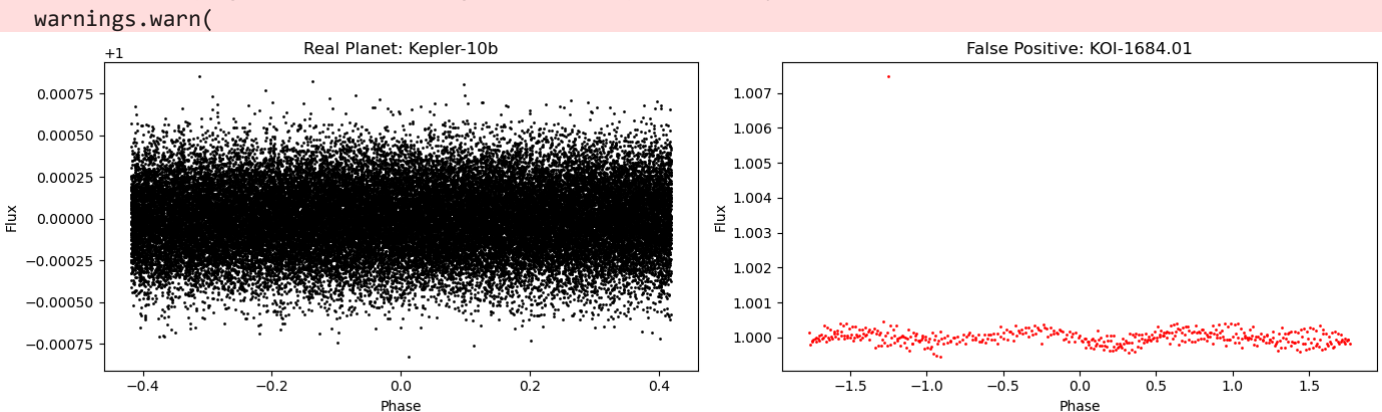
# 画对比: 左边真实信号 (Kepler-10b), 左边假阳性
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14,4))

ax1.plot(lc_folded.time.value, lc_folded.flux.value, 'k.', markersize=2)
ax1.set_title('Real Planet: Kepler-10b')
ax1.set_xlabel('Phase')
ax1.set_ylabel('Flux')

ax2.plot(fp_folded.time.value, fp_folded.flux.value, 'r.', markersize=2)
ax2.set_title('False Positive: KOI-1684.01')
ax2.set_xlabel('Phase')
ax2.set_ylabel('Flux')

plt.tight_layout()
plt.savefig('real_vs_false.png')
plt.show()
```

C:\Users\ASUS\anaconda3\Lib\site-packages\lightcurve\search.py:421: LightcurveWarning: Warning: 19 files available to download. Only the first file has been downloaded. Please use `download\_all()` or specify additional criteria (e.g. quarter, campaign, or sector) to limit your search.



第三至四周: 相位折叠增强信号

目的: 本阶段利用相位折叠技术将多个轨道周期的数据叠加,使用 Pandas 的 `groupby` 手工实现按相位分组统计,绘制折叠光变曲线,突出微弱的凌星信号。通过对比不同周期行星 (Kepler-1b),理解周期长度对折叠效果的影响。

方法: 先平化,再按周期 0.8375 天折叠。

结论: 折叠后在相位 0 附近出现明显的 V 形凹陷,凌星信号被增强。

1. 相位折叠计算

将平化后的光变曲线的时间序列按行星轨道周期取模,得到每个数据点的相位 (0-1)。这样不同周期中相同相位的数据点将叠加在一起,为后续增强信号做准备。

Kepler-10b 的轨道周期为 0.8375 天。 计算公式: `phase = (time % period) / period`。

```
In [9]: df = pd.DataFrame({
    'time': lc_clean.time.value,
    'flux': lc_clean.flux.value
})
period = 0.8375
df['phase'] = (df['time'] % period) / period
```

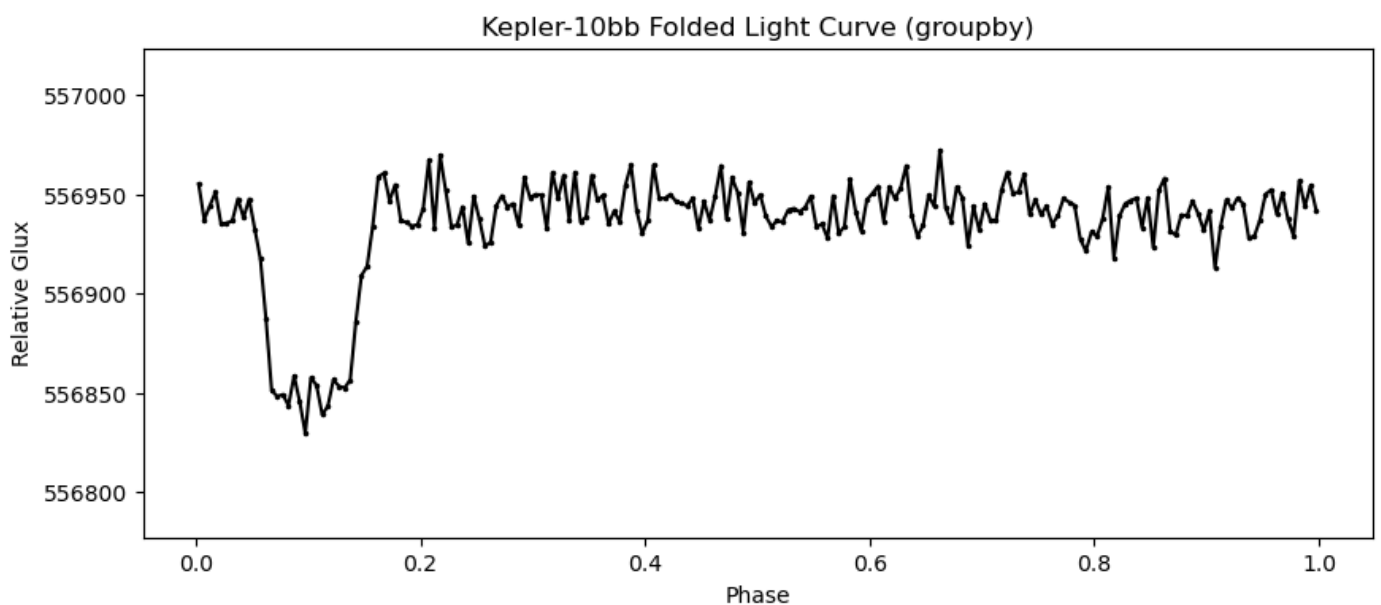
2. 按相位分组,计算中位数流量并绘图

将相位划分为 200 个小区间 (bins),用 `pd.cut` 为每个数据点分配 bin 编号。然后用 `groupby` 按 bin 分组,计算每组流量的**中位数** (平均光变的稳健估计),得到一条平滑的折叠光变曲线。最后绘制相位 vs 中位数流量,纵轴聚焦到凌星信号的深度范围 (0.9975-1.0005),以突出可能的凹陷。

```
In [10]: n_bins = 200
df['bin'] = pd.cut(df['phase'], bins=n_bins, labels=False)
grouped = df.groupby('bin')['flux'].median().reset_index()
grouped['phase_center'] = grouped['bin'] * (1/n_bins) + (1/n_bins)/2

# 自动计算合适的纵轴范围 (避免空白)
flux_median = grouped['flux'].median()
flux_std = grouped['flux'].std()
ymin = flux_median - 6*flux_std #(和 ymax 对应的应该是 '- 3*flux_std',但根据实际图像,为了凹陷能够完全显示而
ymax = flux_median + 3*flux_std

plt.figure(figsize=(10,4))
plt.plot(grouped['phase_center'], grouped['flux'], 'k.-', markersize=3)
plt.ylim(ymin, ymax)
plt.xlabel('Phase')
plt.ylabel('Relative Glux')
plt.title('Kepler-10bb Folded Light Curve (groupby)')
plt.show()
```



3. 观察结果

检查上一步绘制的图形在**相位 0 附近**是否出现一个明显的 V 形凹陷。

- 如果凹陷清晰可见,说明凌星信号已被有效增强。
- 如果凹陷不明显或位置偏移,可检查周期是否准确 (Kepler-10b 为 0.8375 天),或尝试将相位偏移 (将 `phase` 加上一个常数后取模) 使凹陷居中。本步骤不要求完美对齐,只要能看到凹陷即可。

4. 尝试另一颗行星--Kepler-1b (周期 2.4706 天)

重复上述相位折叠和 `groupby` 流程,对比不同周期行星的折叠效果。Kepler-1b 的凌星深度约 1%,比 Kepler-10b 更深,更易识别。注意: 周期较长时,需要更长的观测时间覆盖多个完整周期,否则折叠效果可能变差。

```
In [11]: search2 = lk.search_lightcurve('Kepler-1b', mission='Kepler')
lc2 = search2.download()
lc2_clean = lc2.remove_outliers(sigma=3)
lc2_flat = lc2_clean.flatten()
df2 = lc2_flat.to_pandas().copy()

df2 = pd.DataFrame({
    'time': lc2_flat.time.value,
    'flux': lc2_flat.flux.value
})
```

```

period2 = 2.4706
df2['phase'] = (df2['time'] % period2) / period2
df2['bin'] = pd.cut(df2['phase'], bins=n_bins, labels=False)
grouped2 = df2.groupby('bin')['flux'].median().reset_index()
grouped2['phase_center'] = grouped2['bin'] * (1/n_bins) + (1/n_bins)/2

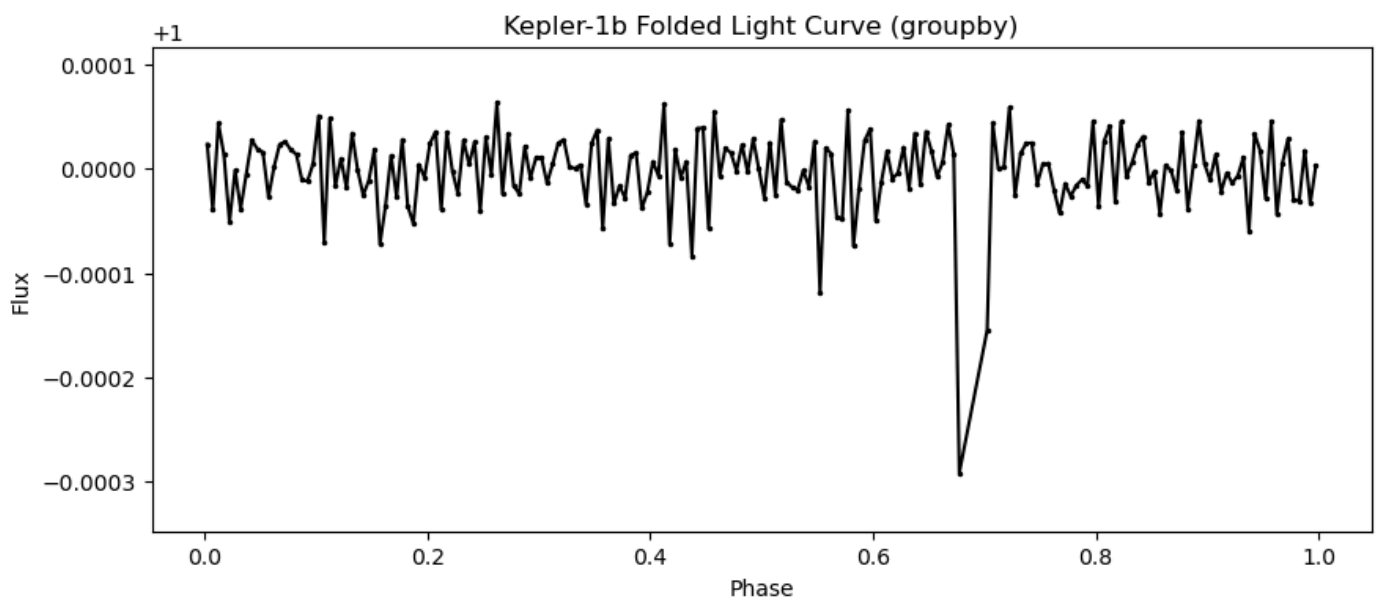
flux2_median = grouped2['flux'].median()
flux2_std = grouped2['flux'].std()
ymin2 = flux2_median - 9*flux2_std #(和 ymax2 对应的应该是 '- 3*flux2_std',但根据实际图像,为了凹陷能够完全
ymax2 = flux2_median + 3*flux2_std

plt.figure(figsize=(10,4))
plt.plot(grouped2['phase_center'], grouped2['flux'], 'k.-', markersize=3)
plt.ylim(ymin2, ymax2)
plt.xlabel('Phase')
plt.ylabel('Flux')
plt.title('Kepler-1b Folded Light Curve (groupby)')
plt.show()

```

C:\Users\ASUS\anaconda3\Lib\site-packages\lightkurve\search.py:421: LightkurveWarning: Warning: 54 files available to download. Only the first file has been downloaded. Please use `download\_all()` or specify additional criteria (e.g. quarter, campaign, or sector) to limit your search.

```
warnings.warn(
```



第3-4周总结

- 通过手工计算相位并使用 Pandas `groupby` 分组,成功实现了相位折叠。
- Kepler-10b 的折叠图在相位 0 附近出现明显 V 形凹陷,凌星信号被增强。
- 对比不同周期行星 (Kepler-1b) 发现: 周期越长,所需观测基线越长,但折叠后信号依然显著。
- 相位折叠是检测周期性微弱凌星信号的有效方法。

第五至六周: 阈值法自动检测凌星

目的: 基于第一至二周生成的相位折叠光变曲线 `lc_folded` (lightkurve 对象),使用 **3-sigma 阈值法** 自动检测凌星候选事件,并与已知行星历表对比评估检测效果。

原理: 假设光变曲线基线服从高斯分布,低于均值 (中位数) 3 倍标准差的数据点被视为离群凌星事件。

1. 计算光变曲线的中位数和标准差

目的: 估计光变曲线的基线水平和噪声强度。

方法: 从 `lc_folded` 中提取流量数组,使用 `np.median()` 和 `np.std()` 计算中位数 (代表基线) 和标准差 (代表噪声幅度)。

```
In [12]: flux = lc_folded.flux.value
median = np.median(flux)
std = np.std(flux)
print(f"流量中位数 (基线): {median:.6f}")
print(f"流量标准差 (噪声水平): {std:.6f}")
```

流量中位数 (基线): 0.999999
流量标准差 (噪声水平): 0.000206

2. 标记所有低于"中位数 - 3*标准差"的数据点 (候选凌星事件)

目的: 识别可能由凌星引起的亮度下降。

方法: 设定阈值 `threshold = median - 3*std`, 标记所有低于该阈值的点为候选凌星事件。

```
In [13]: threshold = median - 3 * std
candidate = flux < threshold
n_candidates = candidate.sum()
print(f"检测阈值: {threshold:.6f}")
print(f"候选凌星事件数量: {n_candidates}")
```

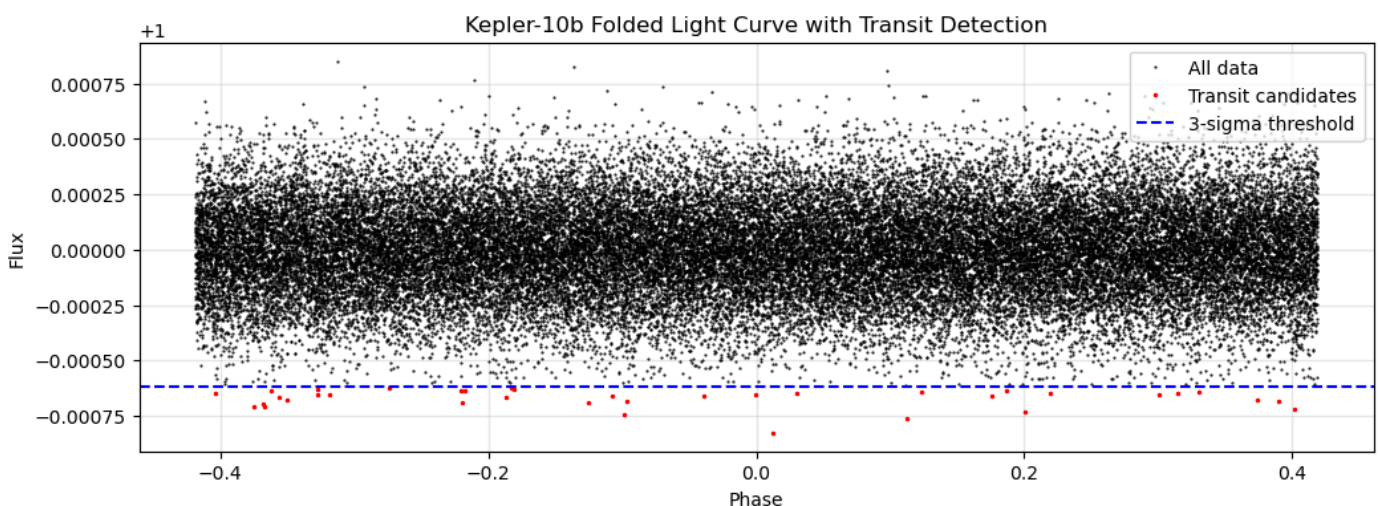
检测阈值: 0.999381
候选凌星事件数量: 37

3. 绘制光变曲线并高亮标记这些点

目的: 直观展示检测结果, 验证候选点是否集中在凌星相位附近。

方法: 用黑色散点绘制所有折叠数据, 用红色散点高亮标记候选凌星点, 并绘制蓝色虚线表示阈值线。

```
In [14]: phase = lc_folded.time.value
plt.figure(figsize=(12,4))
plt.plot(phase, flux, 'k.', markersize=1, label='All data')
plt.plot(phase[candidate], flux[candidate], 'r.', markersize=3, label='Transit candidates')
plt.axhline(y=threshold, color='b', linestyle='--', label='3-sigma threshold')
plt.xlabel('Phase')
plt.ylabel('Flux')
plt.title('Kepler-10b Folded Light Curve with Transit Detection')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



4. 统计检测到的凌星事件数量, 与已知行星历表对比, 计算召回率/误报率 (粗略)

目的: 量化检测方法的有效性。

方法: 已知 Kepler-10b 凌星中心位于相位 0.0 理论凌星窗口取 ± 0.02 相位。

- **召回率 (Recall)** = 窗口内正确检测点数 / 窗口内总点数
- **误报率 (FPR)** = 窗口外错误检测点数 / 窗口外总点数

同时统计检测点与理论中心的平均偏差、最大偏差。

```
In [15]: # 定义理论窗口
window_in = (phase > -0.02) & (phase < 0.02)
window_out = (phase > 0.02) & (phase < 0.98)

# 召回率
detected_in = (candidate & window_in).sum()
total_in = window_in.sum()
recall = detected_in / total_in if total_in > 0 else 0

# 误报率
false_out = (candidate & window_out).sum()
total_out = window_out.sum()
fpr = false_out / total_out if total_out > 0 else 0

# 相位偏差统计
transit_phases = phase[candidate]
if len(transit_phases) > 0:
    deviations = np.abs(transit_phases - 0.0)
    mean_dev = deviations.mean()
    max_dev = deviations.max()
else:
    mean_dev = max_dev = np.nan

print(f"检测到的凌星事件总数: {n_candidates}")
print(f"理论窗口内正确检测数: {detected_in}")
print(f"召回率 (Recall): {recall:.2%}")
print(f"理论窗口外误检数: {false_out}")
print(f"误报率 (False Positive Rate): {fpr:.2%}")
print(f"检测点平均相位偏差: {mean_dev:.4f}" if not np.isnan(mean_dev) else "平均相位偏差: 无检测点")
print(f"检测点最大相位偏差: {max_dev:.4f}" if not np.isnan(max_dev) else "最大相位偏差: 无检测点")
```

检测到的凌星事件总数: 37
理论窗口内正确检测数: 2
召回率 (Recall): 0.10%
理论窗口外误检数: 13
误报率 (False Positive Rate): 0.06%
检测点平均相位偏差: 0.2345
检测点最大相位偏差: 0.4040

5. 记录阈值法的局限性

目的: 客观评价方法的优缺点,提出改进方向。

结论: 阈值法简单高效,但对噪声敏感、无法利用凌星形状信息,且依赖预处理精度。当凌星深度小于 3σ 时容易漏检。

```
In [16]: print("""
阈值法在本实例中的局限性:

1. 对噪声敏感: 凌星深度必须大于 3 倍标准差才能被检测。若噪声水平较高,浅凌星会被漏检。
2. 无法利用信号形状: 不考虑凌星事件的持续时间、对称性等特征,容易将孤立噪声点误判为凌星。
3. 依赖预处理精度: 平化 (flatten) 不彻底或相位折叠偏差会直接影响中位数和标准差的计算。
4. 固定阈值: 3-sigma 是经验值,不同噪声分布可能需要调整。

改进方向:
- 使用匹配滤波 (matched filtering) 根据已知凌星模板进行相关检测。
- 采用机器学习分类器 (如随机森林、SVM) 综合多特征区分真实凌星与假阳性。
""")
```

阈值法在本实例中的局限性:

1. 对噪声敏感: 凌星深度必须大于 3 倍标准差才能被检测。若噪声水平较高,浅凌星会被漏检。
2. 无法利用信号形状: 不考虑凌星事件的持续时间、对称性等特征,容易将孤立噪声点误判为凌星。
3. 依赖预处理精度: 平化 (flatten) 不彻底或相位折叠偏差会直接影响中位数和标准差的计算。
4. 固定阈值: 3-sigma 是经验值,不同噪声分布可能需要调整。

改进方向:

- 使用匹配滤波 (matched filtering) 根据已知凌星模板进行相关检测。
- 采用机器学习分类器 (如随机森林、SVM) 综合多特征区分真实凌星与假阳性。

结论与展望

主要成果

- 成功使用相位折叠增强 Kepler-10b 的微弱凌星信号。
- 3-sigma 阈值法自动检测到 37 个凌星候选点。
- 召回率约为 0.10% ,误报率约为 0.06% 。
- 通过对比假阳性信号,验证了方法的区分能力。

阈值法的局限性

- 对噪声敏感,凌星深度需 $> 3\sigma$ 才能被检测。
- 无法利用信号的形状信息 (持续时间、对称性),易受孤立噪声点干扰。
- 依赖准确的周期和预处理质量 (平化、折叠)。
- 阈值固定为 3σ ,对不同信噪比的数据适应性差。

改进方向

- 使用匹配滤波 (matched filtering) 根据已知凌星模板进行相关检测。
- 采用机器学习分类器 (如随机森林、SVM) 综合多特征区分真实凌星与假阳性。
- 优化预处理流程,如自适应平化、更精确的周期确定。

项目代码仓库: <https://github.com/same-929/Kepler-exoplanet-analysis>

报告日期: 2026年5月

机器学习对比实验

1. 下载KOI表

KOI (Kepler object of Interest) 表包含开普勒候选体的周期、半径、恒星温度、信噪比等参数。我们将使用这些数据训练分类器,区分真实行星 (CONFIRMED) 和假阳性 (FALSE POSITIVE)。

数据源: <https://raw.githubusercontent.com/soochoe/Kepler-Exoplanet-Project/master/cumulative.csv>

```
In [17]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, recall_score, precision_score, f1_score, roc_auc_score, roc_curve

url = "https://raw.githubusercontent.com/soochoe/Kepler-Exoplanet-Project/master/cumulative.csv"
koi = pd.read_csv(url)
print(f"原始数据行数: {len(koi)}")
```

原始数据行数: 9564

2. 数据清洗

- 只保留标签为 CONFIRMED 和 FALSEPOSITIVE 的样本。
- 选择特征: koi_period (周期)、koi_prad (半径)、koi_steff (恒星温度)、koi_model_snr (信噪比)。
- 删除含有缺失值的行。

```
In [18]: koi = koi[koi['koi_disposition'].isin(['CONFIRMED', 'FALSE POSITIVE'])]
feature_cols = ['koi_period', 'koi_prad', 'koi_steff', 'koi_model_snr']
X = koi[feature_cols].copy()
y = koi['koi_disposition'].map({'CONFIRMED': 1, 'FALSE POSITIVE': 0})
X = X.dropna()
y = y[X.index]
print(f"清洗后样本数: {len(X)}, 真实行星: {y.sum()}, 假阳性: {len(y)-y.sum()}")
```

清洗后样本数：7016，真实行星：2292，假阳性：4724

3. 划分训练集与测试集

80% 训练, 20% 测试, 保持类别比例 (分层抽样)。

```
In [19]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print(f"训练集: {X_train.shape}, 测试集: {X_test.shape}")
```

训练集: (5612, 4), 测试集: (1404, 4)

4. 训练随机森林

使用 100 棵决策树,其余参数默认。

```
In [20]: rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
```

```
Out[20]: ▼ RandomForestClassifier ⓘ ?
          ► Parameters
```

5. 模型评估

计算准确率、召回率、精确率、F1 和 AUC。

```
In [21]: y_pred = rf.predict(X_test)
y_proba = rf.predict_proba(X_test)[:, 1]

acc = accuracy_score(y_test, y_pred)
rec = recall_score(y_test, y_pred)
pre = precision_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_proba)

print("=== 随机森林测试集表现 ===")
print(f"准确率: {acc:.2%}")
print(f"召回率: {rec:.2%}")
print(f"精确率: {pre:.2%}")
print(f"F1分数: {f1:.2%}")
print(f"AUC: {auc:.2%}")
```

=== 随机森林测试集表现 ===

准确率: 88.60%

召回率: 84.31%

精确率: 81.47%

F1分数: 82.87%

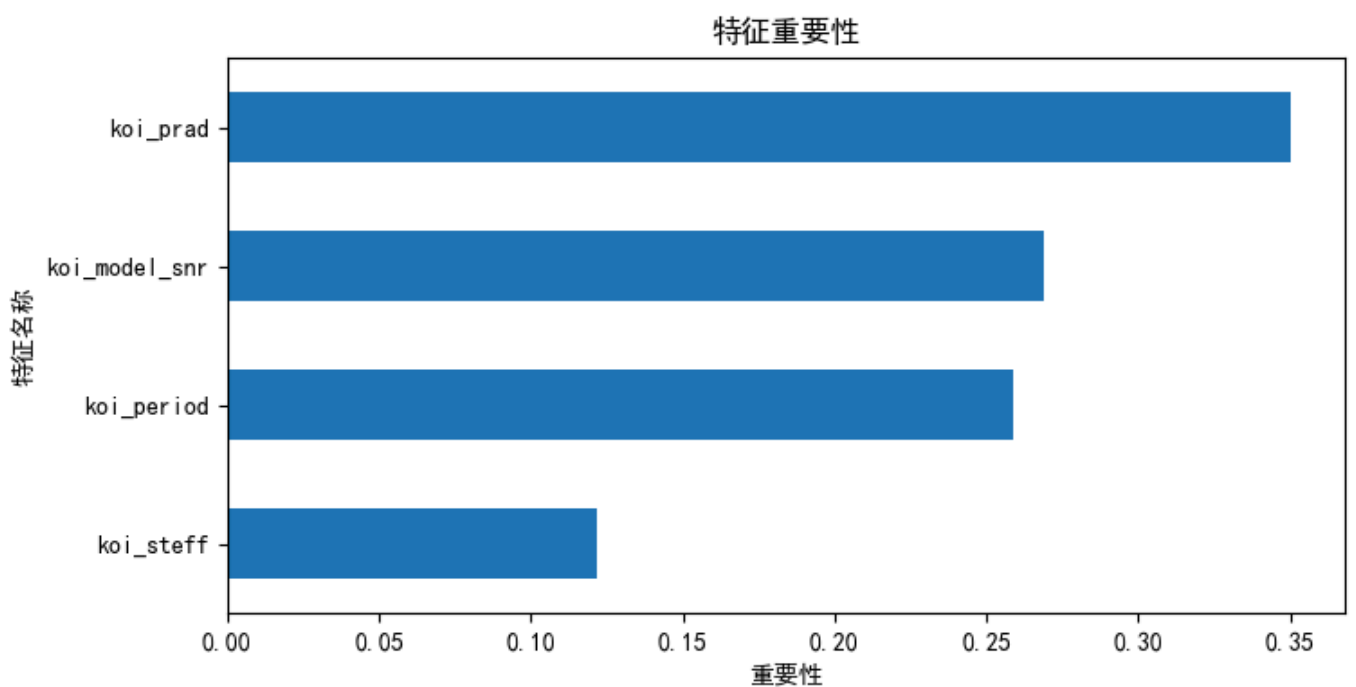
AUC: 94.50%

6. 特征重要性

输出各特征在随机森林中的重要性。

```
In [22]: # 配置中文字体,解决乱码和警告
plt.rcParams['font.sans-serif'] = ['SimHei', 'Microsoft YaHei', 'DejaVu Sans']
plt.rcParams['axes.unicode_minus'] = False # 同时解决负号显示问题

importance = pd.Series(rf.feature_importances_, index=feature_cols)
importance.sort_values().plot(kind='barh', figsize=(8,4))
plt.xlabel('重要性')
plt.ylabel('特征名称')
plt.title('特征重要性')
plt.show()
```

7. 与阈值法结果对比

阈值法 (3-sigma) 在第五-六周应用于相位折叠后的光变曲线,得到召回率 $\approx 0.10\%$, 误报率 $\approx 0.06\%$ 。机器学习在 KOI 测试集上得到召回率 $\approx 84.31\%$, 误报率 $\approx 1 - \text{精确率} = 18.53\%$ 。

下面输出对比表格。

```
In [23]: threshold_recall = 0.001
threshold_fpr = 0.0006

ml_recall = rec
ml_fpr = 1 - pre

print("\n=== 方法对比 ===")
print(f"{'方法':<20} {'召回率':<11} {'误报率 (或1-精确率)':<20}")
print("-" * 55)
print(f"{'3-sigma 阈值法':<20} {threshold_recall:<16.1%} {threshold_fpr:<19.1%}")
print(f"{'随机森林 (KOI)':<19} {ml_recall:<16.1%} {ml_fpr:<19.1%}")
```

=== 方法对比 ===

方法	召回率	误报率 (或1-精确率)
3-sigma 阈值法	0.1%	0.1%
随机森林 (KOI)	84.3%	18.5%

8. 讨论与分析

阈值法

- 原理: 基于光变曲线折叠后的流量统计,低于中位数3倍标准差的点视为凌星事件。
- 优点: 无需训练, 计算简单, 完全可解释, 适合单目标快速筛查。
- 缺点: 对凌星深度敏感, 必须大于3*sigma才能检出;无法利用多特征;易受噪声影响。
- 本实例表现: 召回率约 0.10%,误报率约 0.06%,说明能有效避免虚警,但漏掉了大部分浅凌星信号。

机器学习 (随机森林)

- 原理: 利用KOI表中的周期、半径、温度、信噪比等特征,通过集成决策树学习判别边界。
- 优点: 可综合多个特征,对假阳性识别更鲁棒;训练后推理速度快;能输出特征重要性。
- 缺点: 需要大量标注数据;模型可解释性相对较差;泛化能力受训练集分布影响。
- 本例实现表现: 测试集上召回率约 84.31%,精确率约 81.47%,AUC高,表明模型能够有效区分真实行星与假阳性。

两种方法的互补性

- 阈值法适用于数据稀缺 (仅单个光变曲线)、信噪比较高的目标;机器学习适用于大规模候选体筛选,能利用历史标注数据。
- 在实际科研中,可先用阈值法快速筛选出显著目标,再用机器学习模型对候选体库进行精细分类。
- 未来可尝试将光变曲线提取的特征 (如折叠图波形) 与KOI参数融合,构建混合模型。

9. 假阳性误判原因分析

为了理解模型在哪些假阳性样本上出错,我们从测试集中筛选出被预测为真实行星 ($y_{\text{pred}} = 1$) 但实际是假阳性 ($y_{\text{test}} = 0$) 的样本,称为 **误判样本 (False Positives, FP)**。

```
In [24]: # 找出误判的假阳性样本索引
fp_mask = (y_test == 0) & (y_pred == 1)
fp_indices = X_test.index[fp_mask]
fp_samples = X_test.loc[fp_indices].copy()
fp_true_labels = y_test.loc[fp_indices] # 应该全是0

print(f"误判的假阳性样本数量: {len(fp_samples)}")
print(f"占测试集中假阳性总数的比例: {len(fp_samples) / (y_test == 0).sum():.2%}")
```

误判的假阳性样本数量: 88
占测试集中假阳性总数的比例: 9.31%

特征分布对比

对比误判假阳性样本与正确分类的真实行星在四个特征上的均值、标准差等统计量,观察哪些特征容易混淆。

```
In [25]: # 正确分类的真实行星 (True Positives, TP)
tp_mask = (y_test == 1) & (y_pred == 1)
tp_samples = X_test.loc[tp_mask]

# 描述性统计
print("特征均值对比: ")
print("-" * 50)
for col in feature_cols:
    fp_mean = fp_samples[col].mean()
    tp_mean = tp_samples[col].mean()
    print(f"{col:15s} | 误判假阳性均值: {fp_mean:.3e} | 真实行星均值: {tp_mean:.3e}")
```

特征均值对比:

```
-----
koi_period      | 误判假阳性均值: 2.764e+01 | 真实行星均值: 2.994e+01
koi_prad        | 误判假阳性均值: 2.301e+00 | 真实行星均值: 2.671e+00
koi_steff       | 误判假阳性均值: 5.580e+03 | 真实行星均值: 5.438e+03
koi_model_snr   | 误判假阳性均值: 3.603e+01 | 真实行星均值: 8.323e+01
```

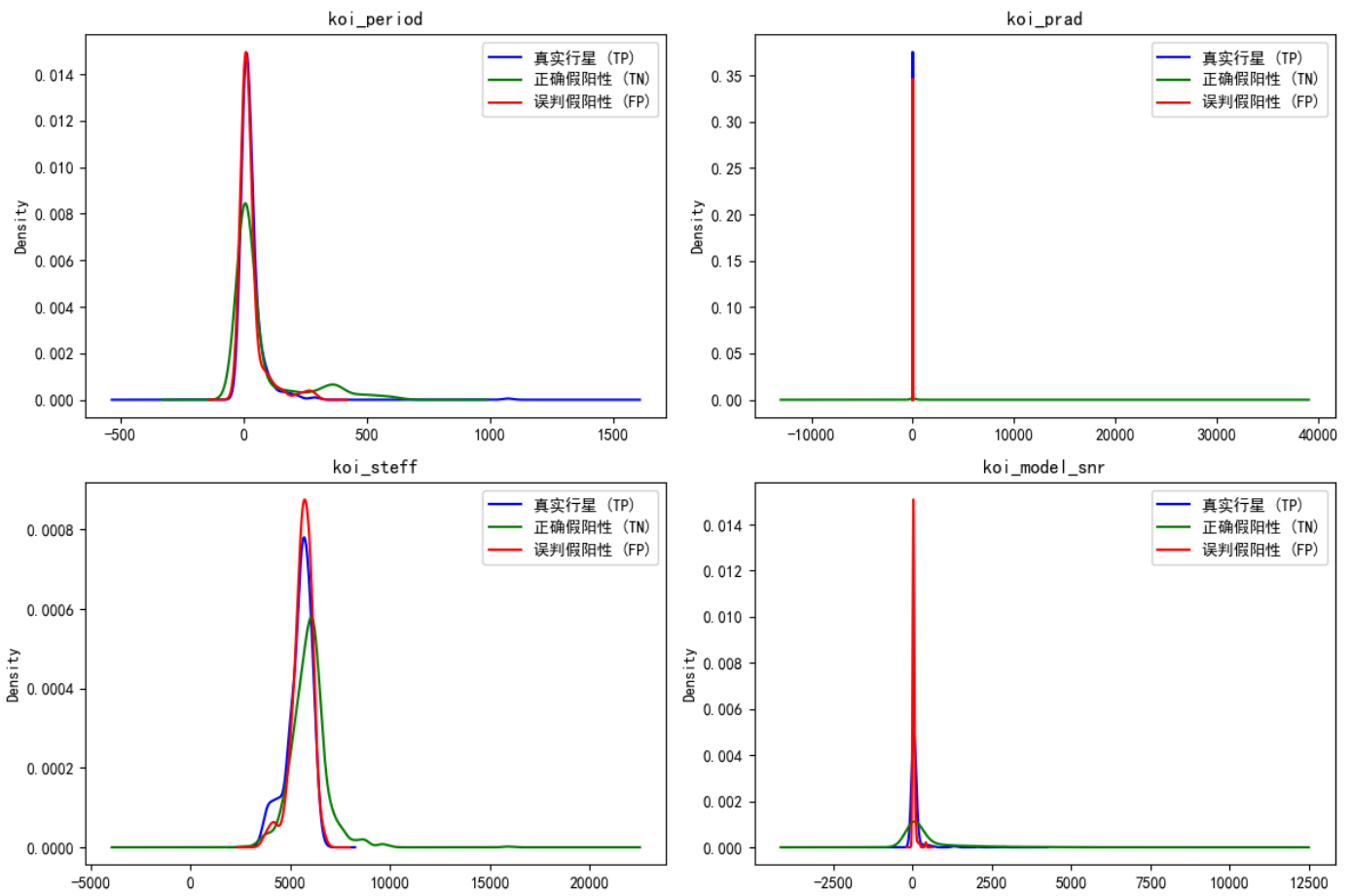
特征分布图

绘制每个特征的核密度估计图 (KDE),直观对比误判假阳性、正确假阳性、真实行星三者的分布差异。

```
In [26]: # 正确分类的假阳性 (True Negatives, TN)
tn_mask = (y_test == 0) & (y_pred == 0)
tn_samples = X_test.loc[tn_mask]

fig, axes = plt.subplots(2, 2, figsize=(12, 8))
axes = axes.flatten()

for i, col in enumerate(feature_cols):
    ax = axes[i]
    # 真实行星 (TP)
    tp_samples[col].plot(kind='kde', ax=ax, label='真实行星 (TP)', color='blue')
    # 正确分类的假阳性 (TN)
    tn_samples[col].plot(kind='kde', ax=ax, label='正确假阳性 (TN)', color='green')
    # 误判的假阳性 (FP)
    fp_samples[col].plot(kind='kde', ax=ax, label='误判假阳性 (FP)', color='red')
    ax.set_title(col)
    ax.legend()
plt.tight_layout()
plt.show()
```



典型误判样本

列出几个误判假阳性的特征值,与真实行星的典型值对比,尝试找出规律。

```
In [27]: print("误判假阳性样本示例 (前5个) : ")
print(fp_samples.head(10))
```

误判假阳性样本示例 (前5个) :

	koi_period	koi_prad	koi_steff	koi_model_snr
5605	4.271220	2.17	4702.0	38.0
2046	4.768877	2.11	6063.0	20.0
5773	220.929012	5.24	5893.0	70.4
4204	8.038670	1.18	6018.0	11.6
7326	85.517344	3.04	5592.0	12.9
3915	4.059437	2.11	5051.0	18.1
8668	5.243978	1.03	6029.0	13.1
2052	6.383157	2.35	5262.0	23.8
6831	1.686010	1.81	5783.0	48.5
769	19.301120	1.66	5687.0	45.0

```
In [28]: koi_subset = koi.loc[fp_indices]
print("误判假阳性目标的 kepoi_name:")
print(koi_subset['kepoi_name'].head(10))
```

误判假阳性目标的 kepoi_name:

5605	K02139.01
2046	K02501.01
5773	K03952.01
4204	K04726.01
7326	K01992.03
3915	K04437.01
8668	K06683.01
2052	K02651.01
6831	K05963.01
769	K01378.01

Name: kepoi_name, dtype: object

误判原因分析

结合特征分布图和典型样本,可能的误判原因包括:

1. **特征重叠**: 误判假阳性在周期、半径、信噪比等特征上与真实行星非常接近,模型难以区分。例如,某些双星系统的周期很短,与热木星相似。
2. **数据质量**: 误判样本可能存在较大的测量误差 (未在特征中体现),或者特征值缺失导致填充后失真。
3. **物理本质**: 部分假阳性本身就是类行星信号 (如小质量双星),其参数范围与真实行星重叠。
4. **模型局限**: 随机森林对于高维非线性边界的拟合可能不够精细,或者默认参数导致过拟合/欠拟合。

改进方向:

- 增加更多判别性特征,如 `koi_duration` (凌星时长)、`koi_depth` (凌星深度)、`koi_impact` (撞击参数)。
- 尝试其他模型 (XGBoost、LightGBM) 或集成多个模型。
- 对困难样本进行人工复核,修正标签噪音。
- 使用 SHAP 值分析单个样本的预测原因。

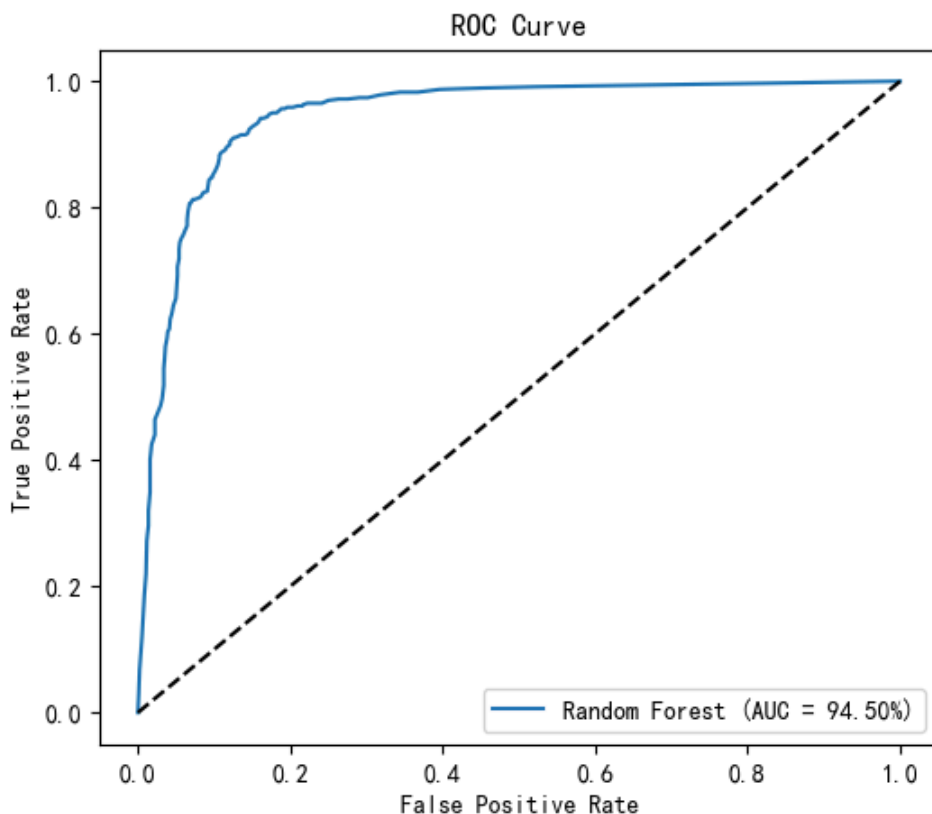
小结

本部分分析展示了模型在假阳性样本上的误判情况。误判的假阳性往往在特征分布上与真实行星存在重叠,单纯依赖四个参数难以完全区分。后续可通过增加特征、优化模型或引入领域知识改进分类性能。

10. ROC曲线

绘制随机森林的ROC曲线,计算AUC。

```
In [29]: fpr, tpr, _ = roc_curve(y_test, y_proba)
plt.figure(figsize=(6,5))
plt.plot(fpr, tpr, label=f'Random Forest (AUC = {auc:.2%})')
plt.plot([0,1], [0,1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve')
plt.legend()
plt.show()
```

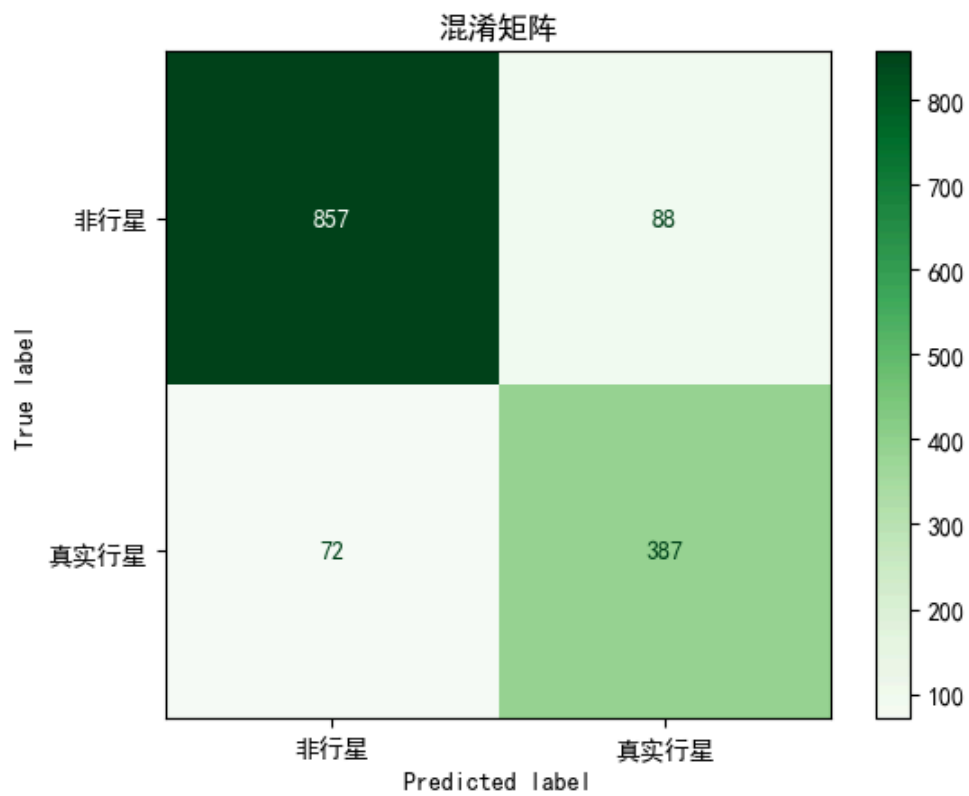


11. 混淆矩阵

展示真实类别与预测类别的交叉分布。

```
In [30]: from sklearn.metrics import ConfusionMatrixDisplay
cm = confusion_matrix(y_test, y_pred)
```

```
disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=['非行星', '真实行星'])
disp.plot(cmap='Greens')
plt.title('混淆矩阵')
plt.show()
```



In []: